

Programmation I : TD/TP4

(Pointeurs et Allocation

dynamique)

Exercice1 :

1. `*p + 2` :

- `p` pointe sur le premier élément de `A`, donc `*p` est égal à `A[0]`, qui est `12`.
- `*p + 2` signifie `12 + 2`, donc cela donne `14`.

2. `*(p + 2)` :

- `p + 2` signifie que nous décalons le pointeur `p` de deux positions dans le tableau `A`.
- `*(p + 2)` est donc équivalent à `A[2]`, ce qui vaut `27`.

3. `*(p + (*p - 9))` :

- Ici, `*p` est encore `A[0]`, soit `12`.
- Donc, `*p - 9` est égal à `12 - 9 = 3`.
- `p + 3` pointe alors vers `A[3]`, donc `*(p + 3)` est équivalent à `A[3]`, qui vaut `42`.

Exercice2 :

	<u>A</u>	<u>B</u>	<u>C</u>	<u>P1</u>	<u>P2</u>
Init.	1	2	3	/	/
P1=&A	1	2	3	&A	/
P2=&C	1	2	3	&A	&C
*P1=(*P2)++	3	2	4	&A	&C
P1=P2	3	2	4	&C	&C
P2=&B	3	2	4	&C	&B
*P1--=*P2	3	2	2	&C	&B
++*P2	3	3	2	&C	&B
P1=*P2	3	3	6	&C	&B
A=++*P2**P1	24	4	6	&C	&B
P1=&A	24	4	6	&A	&B
*P2=*P1/=*P2	6	6	6	&A	&B

Exercice3 :

```
#include <stdio.h>

#include <stdlib.h>

main()
{
    int N;

    float T[100],AIDE,*P1, *P2;

    printf(" Entrez la dimension du tableau \n");

    scanf("%d", &N );

    for (P1=T; P1<T+N; P1++)
    {
        printf("Entrez T[%d]: \n", P1-T);

        scanf("%f", P1);

    }

    /* Affichage du tableau avant inversion*/

    for (P1=T; P1<T+N; P1++)

        printf("%f \t", *P1);

    printf("\n");

    /* Inverser tableau */

    for (P1=T,P2=T+(N-1); P1<P2; P1++,P2--)

    {

        AIDE = *P1;

        *P1 = *P2;

        *P2 = AIDE;

    }

    for (P1=T; P1<T+N; P1++)

        printf("%f \t", *P1);

    printf("\n");

}
```

Exercice4 :

```
#include <stdio.h>

main()
{
    int N, A[100], *P1, *P2;

    printf(" Entrez N \n ");
    scanf("%d", &N );

    for (P1=A; P1<A+N; P1++)
    {
        printf("Entrez A[%d] : ", P1-A);
        scanf("%d", P1);
    }

    /* Affichage du tableau avant suppression */

    for (P1=A; P1<A+N; P1++)
        printf("%d \t", *P1);

    printf("\n");

    P2=A;

    for (P1=A; P1<A+N; P1++)
        if (*P1 != 0)
        {
            *P2 = *P1;
            P2++;
        }

    /* Nouvelle dimension de A */

    N = P2-A;

    /* Affichage du tableau apres suppression */

    for (P1=A; P1<A+N; P1++)
        printf("%d \t ", *P1);

    printf("\n");
}
```

Exercice 5:

```
#include <stdio.h>
int main(){
    /* Déclarations */
    int A[50][50]; /* matrice donnée */
    int B[50][50]; /* matrice donnée */
    int C[50][50]; /* matrice résultat */
    int N, M, P; /* dimensions des matrices */
    int I, J, K; /* indices courants */
    /* Saisie des données */
    printf("*** Matrice A ***\n");
    printf("Nombre de lignes de A (max.50) : ");
    scanf("%d", &N);
    printf("Nombre de colonnes de A (max.50) : ");
    scanf("%d", &M);
    for (I=0; I<N; I++){
        for (J=0; J<M; J++){
            printf("Elément[%d][%d] : ", I, J);
            scanf("%d", (int *)A+I*50+J);
        }
    }
    printf("*** Matrice B ***\n");
    printf("Nombre de lignes de B : %d\n", M);
    printf("Nombre de colonnes de B (max.50) : ");
    scanf("%d", &P);
    for (I=0; I<M; I++){
        for (J=0; J<P; J++){
            printf("Elément[%d][%d] : ", I, J);
            scanf("%d", (int *)B+I*50+J);
        }
    }
    /* Affichage des matrices */
    printf("Matrice donnée A :\n");
    for (I=0; I<N; I++){
        for (J=0; J<M; J++){
            printf("%7d", *((int *)A+I*50+J)); printf("\n");
        }
    }
    printf("Matrice donnée B :\n");
    for (I=0; I<M; I++){
        for (J=0; J<P; J++){
            printf("%7d", *((int *)B+I*50+J));
        }
        printf("\n");
    }
}
```

Exercice 6:

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
main()
```

```
{ int i,N,M,*A,*B,*PA,*PB;
```

```
    printf ("Entrez n et m \n");
```

```
    scanf("%d%d",&N,&M);
```

```
    A=(int*)malloc ((N+M)*sizeof(int));
```

```
    B=(int*) malloc (M*sizeof(int));
```

```
    for (PA=A;PA<A+N;PA++)
```

```
        { printf("Entrez A[%d]\n",PA-A);
```

```
            scanf("%d",PA);
```

```
        }
```

```
    for (PB=B;PB<B+M;PB++)
```

```
        { printf("Entrez B[%d]\n",PB-B);
```

```
            scanf("%d",PB);
```

```
        }
```

```
// Affichage des tableaux A et B
```

```
for (PA=A;PA<A+N;PA++)
```

```
    printf("%d \t",*PA);
```

```
    printf("\n");
```

```
for (PB=B;PB<B+M;PB++)
```

```
    printf("%d \t",*PB);
```

```
    printf("\n");
```

```
//Ajout de B à la fin de A
```

```
PB=B;
```

```
for (PA=A+N;PA<A+N+M;PA++)
```

```
    { *PA=*PB;
```

```
      PB++;
```

```
    }
```

```
for (PA=A;PA<A+N+M;PA++)
```

```
    printf("%d \t",*PA);
```

```
    printf("\n");
```

```
}
```

Exercice 7:

```

#include<stdio.h>
#include<stdlib.h>

int main() {
    int n, i;
    double *A, *B, s=0.;
    printf("saisir la taille du tableau");
    scanf("%d",&n);
    /*l'allocation de l'espace mémoire dynamiquement*/
    A = (double*)malloc(n*sizeof(double));
    B = (double*)malloc(n*sizeof(double));

    for( i= 0; i < n; i++)
    {
        printf("Entrez T[%d]: \n", i);
        scanf("%lf",A+i);
    }

    for( i= 0; i < n; i++)
    {
        printf("Entrez T[%d]: \n", i);
        scanf("%lf",B+i);
    }

    for( i= 0; i < n; i++)
        s = s + *(A+i)**(B+i);

    printf("le produit scalaire de A et B : %lf",s);

    /*libération de l'espace mémoire*/
    free(A);
    free(B);
    return 0;
}

```

Exercice8:

```

#include<stdio.h>
#include<stdlib.h>
int main() {
    int n; /* nombre d'entiers à saisir */
    int *tab; /* le tableau de stockage */
    int i; /* compteur */

    /* Demande du nombre d'entiers à saisir */
    printf("Entrer n=");
    scanf("%d", &n);

    /* Réserve l'espace pour le tableau */
    tab=(int*)malloc(n*sizeof(int));

    /* Saisir les n entiers */

    for(i=0; i<n; i++)
    {
        printf("element : ", i);
        scanf("%d", &tab[i]);
    }

    /* Réserve un espace de plus pour ajouter les 3 entiers qui seront saisis */

    realloc(tab, (n+3)*sizeof(int));

    for(i=n; i<n+3; i++){
        printf(" un entier:", i);
        scanf("%d", &tab[i]);
    }
    printf("on affiche toutes les valeurs de tab\n ");
    for(i=0; i<n+3; i++)
        printf("%d\t", tab[i]);
}

```

